

SISTEMA INTEGRAL DE RESERVAS PARA ESPACIOS DE COWORKING

INFORMACIÓN DEL DOCUMENTO

Título: Sistema Integral de Reservas para Coworking

Fecha de Creación: 22 de Junio de 2026

Autor: Carlos Alberto Escamilla Mariño

Documento: 1091354397

RESUMEN EJECUTIVO

Este documento constituye la especificación técnica integral del "Sistema Integral de Reservas para Espacios de Coworking", un proyecto de software empresarial diseñado para revolucionar la forma en que se gestionan y reservan espacios de trabajo flexibles. El sistema proporciona una plataforma robusta, escalable y segura que facilita la interacción entre usuarios finales y administradores de espacios de coworking.

La arquitectura propuesta implementa patrones modernos de ingeniería de software incluyendo microservicios, event-driven architecture, y cloud-native design. El proyecto está estructurado siguiendo metodología ágil con iteraciones de 2 semanas, permitiendo flexibilidad y respuesta rápida a cambios.

Métricas Clave del Proyecto

Métrica	Valor	Descripción
Duración Estimada	21 semanas	Desde inicio hasta producción
Equipo Requerido	8 personas	1 PM, 4 Dev, 1 QA, 1 DevOps, 1 Scrum Master
Presupuesto Total	\$286,000	Año 1 incluyendo infraestructura
ROI Proyectado	102.8%	Retorno en primer año
Período de Recuperación	5.9 meses	Break-even point
Disponibilidad Target	99.5%	SLA para ambiente de producción
Cobertura de Pruebas	80%+	Mínimo requerido de código cubierto
Usuarios Concurrentes	500	Capacidad de usuario simultáneos
Latencia P95	< 2 segundos	Objetivo de tiempo de respuesta

Objetivos Estratégicos

- Crear plataforma centralizada para gestión de reservas de espacios coworking

- Mejorar experiencia del usuario reduciendo tiempo de reserva de 5 minutos a 30 segundos
- Implementar sistema de pagos automatizado y seguro con múltiples métodos
- Proporcionar análisis en tiempo real de ocupación y disponibilidad
- Escalar a 10,000+ usuarios activos en el primer año de operación
- Mantener cumplimiento con regulaciones de privacidad y seguridad de datos (GDPR, CCPA)

TABLA DE CONTENIDOS

Contenido

RESUMEN EJECUTIVO	2
1. REQUERIMIENTOS DE SOFTWARE.....	7
1.1 Documento SRS - Especificación de Requisitos de Software	7
1.2 Matriz de Trazabilidad de Requisitos	9
1.3 Modelos de Casos de Uso Detallados	10
2. ARQUITECTURA DE SOFTWARE.....	13
2.1 Documento SAD - Software Architecture Description.....	13
2.2 Diagrama de Contenedores - C4 Model Nivel 2	15
2.3 Registro de Decisiones Arquitectónicas (ADR)	16
3. DISEÑO DE SOFTWARE.....	19
3.1 Diagrama de Entidad-Relación y Modelo de Datos.....	19
3.2 Especificación de APIs REST - OpenAPI/Swagger.....	22
3.3 Prototipos UI/UX y Flujos de Usuario.....	23
4. CONSTRUCCIÓN DE SOFTWARE.....	25
4.1 Estructura del Código Fuente.....	25
4.2 Estándares de Codificación y Calidad	28
4.3 Scripts de Migración de Base de Datos.....	28
5. PRUEBAS DE SOFTWARE	31
5.1 Plan de Pruebas - Estrategia Integral.....	31
5.2 Casos de Prueba Críticos	31
5.3 Reporte de Cobertura y Métricas de Prueba.....	33
6. OPERACIONES DE INGENIERÍA DEL SOFTWARE.....	34
6.1 Infraestructura como Código (IaC)	34

6.2 Pipelines de CI/CD Automatizados.....	34
6.3 Monitoreo y Alertas.....	34
7. MANTENIMIENTO DE SOFTWARE.....	35
7.1 Manual de Operaciones.....	35
7.2 Plan de Parches	35
7.3 Deuda Técnica Identificada	35
8. GESTIÓN DE CONFIGURACIÓN DEL SOFTWARE.....	36
8.1 Estrategia de Ramas - Gitflow	36
8.2 Release Notes	36
8.3 Inventario de Artefactos.....	36
9. GESTIÓN DE INGENIERÍA DEL SOFTWARE	37
9.1 Cronograma Detallado	37
9.2 Matriz de Riesgos Completa.....	37
10. MODELOS Y MÉTODOS DE INGENIERÍA DEL SOFTWARE.....	38
10.1 Historias de Usuario	38
10.2 Domain Driven Design.....	38
11. PROCESO DE INGENIERÍA DEL SOFTWARE.....	39
11.1 Definición de Listo (DoR) y Hecho (DoD).....	39
12. CALIDAD DEL SOFTWARE	40
12.1 Análisis Estático con SonarQube	40
12.2 Métricas de Calidad.....	40
13. SEGURIDAD DEL SOFTWARE.....	41
13.1 Modelo de Amenazas STRIDE.....	41
13.2 Análisis SAST/DAST	41
14. ECONOMÍA DE INGENIERÍA DEL SOFTWARE	42
14.1 Análisis de ROI	42
APÉNDICES.....	43

APÉNDICE A: Glosario de Términos Técnicos	43
APÉNDICE B: Referencias y Estándares Aplicados.....	44
APÉNDICE C: Checklist de Implementación	44

1. REQUERIMIENTOS DE SOFTWARE

La fase de requerimientos constituye el cimiento fundamental del proyecto. Esta sección documenta de manera exhaustiva todos los requisitos funcionales, no funcionales y de negocio que guiarán el desarrollo, implementación y validación del Sistema de Reservas para Coworking.

1.1 Documento SRS - Especificación de Requisitos de Software

El Documento SRS (Software Requirements Specification) proporciona una especificación formal y detallada de todos los requisitos del sistema. Actúa como contrato vinculante entre stakeholders, clientes, desarrolladores y equipo de aseguramiento de calidad, estableciendo expectativas claras y medibles.

Alcance del Proyecto: El sistema abarcará la gestión integral de reservas de espacios de coworking, incluyendo autenticación de usuarios, búsqueda y filtrado de espacios, reserva y confirmación, procesamiento de pagos, notificaciones, generación de reportes y administración de espacios.

REQUISITOS FUNCIONALES CRÍTICOS DEL SISTEMA:

ID	Requisito Funcional	Descripción Detallada	Prioridad
RF-001	Registro de Usuarios	Permitir registro de nuevos usuarios con validación de email y datos personales	CRÍTICA
RF-002	Autenticación Segura	Implementar autenticación mediante JWT con refresh tokens y 2FA opcional	CRÍTICA
RF-003	Búsqueda de Espacios	Búsqueda avanzada con múltiples filtros (fecha, ubicación, capacidad, amenidades)	CRÍTICA
RF-004	Reserva de Espacios	Crear reserva confirmando disponibilidad en tiempo real y período específico	CRÍTICA

RF-005	Confirmación por Email	Envío automático de confirmación con código de acceso y detalles de reserva	ALTA
RF-006	Cancelación de Reservas	Permitir cancelación con políticas de reembolso según días de anticipación	ALTA
RF-007	Historial de Reservas	Mostrar historial completo con filtros por estado, fecha y espacio	ALTA
RF-008	Gestión de Espacios	CRUD completo de espacios, amenidades, capacidad y configuración de horarios	ALTA
RF-009	Procesamiento de Pagos	Integración con múltiples pasarelas (Stripe, PayPal) con manejo de errores	CRÍTICA
RF-010	Reportes de Ocupación	Generación de reportes con ocupación, ingresos, tendencias de reserva	MEDIA

REQUISITOS NO FUNCIONALES CRÍTICOS:

- RNF-001: Rendimiento - El sistema debe responder en < 2 segundos (P95) para 95% de solicitudes
- RNF-002: Escalabilidad - Soportar mínimo 500 usuarios concurrentes sin degradación notable
- RNF-003: Disponibilidad - Mantener SLA de 99.5% de uptime mensual en producción
- RNF-004: Responsividad - Plataforma completamente responsiva en dispositivos móviles (iOS/Android)
- RNF-005: Seguridad - Encriptación TLS 1.2+ en tránsito, encriptación en reposo para datos sensibles
- RNF-006: Cumplimiento Legal - Conformidad con GDPR, CCPA y leyes locales de protección de datos
- RNF-007: Mantenibilidad - Código limpio con cobertura de pruebas ≥80% y documentación completa

- RNF-008: Compatibilidad - Soporte para navegadores Chrome, Firefox, Safari, Edge últimas 2 versiones
- RNF-009: Recoverabilidad - RTO de 4 horas, RPO de 1 hora en caso de desastre
- RNF-010: Auditoría - Log completo de cambios, accesos y transacciones por requisitos de cumplimiento

1.2 Matriz de Trazabilidad de Requisitos

La matriz de trazabilidad establece una conexión biunívoca entre requisitos de negocio, especificaciones técnicas, código implementado y casos de prueba. Garantiza que cada requisito sea implementado, probado y validado correctamente, evitando requisitos olvidados o no implementados.

Estructura de la Matriz: La matriz utiliza un formato de tabla cruzada donde cada fila representa un requisito único identificado con código (RF-XXX para funcionales, RNF-XXX para no funcionales) y cada columna proporciona información sobre su estado de implementación.

Re q ID	Descripción	Componente Código	Caso Prue ba	Estado	Priorid ad	Valida do
RF-001	Registro usuarios	UserService.register()	CT-001, CT-002	Completo	CRÍTICA	Sí
RF-002	Autenticación	AuthService.login()	CT-003, CT-004, CT-005	Completo	CRÍTICA	Sí
RF-003	Búsqueda espacios	SpaceService.search()	CT-006, CT-007, CT-008	En Desarrollo	CRÍTICA	Parcial

RF-004	Crear reserva	ReservationService.create()	CT-009, CT-010	En Desarrollo	CRÍTICA	No
RNF-001	Latencia P95 < 2s	CacheLayer, QueryOptimization	CT-PERF-001	En Desarrollo	ALTA	No

1.3 Modelos de Casos de Uso Detallados

Los casos de uso describen narrativamente cómo los actores (usuarios, sistemas externos) interactúan con el sistema para lograr objetivos específicos. Proporcionar una visión clara de funcionalidad desde perspectiva de usuario final.

CASO DE USO PRINCIPAL: CU-001 - Realizar una Reserva Completa de Espacio

ID: CU-001

Nombre: Realizar Reserva de Espacio de Coworking

Descripción: Flujo completo donde usuario autenticado busca espacios disponibles, selecciona uno, completa pago y recibe confirmación.

Actores Principales:

- Usuario Registrado: Persona que desea reservar espacio
- Sistema de Reservas: Aplicación que gestiona disponibilidad
- Pasarela de Pago: Servicio tercero para procesar transacción

Actores Secundarios:

- Servidor de Email: Envío de confirmación
- Sistema de Notificaciones: Alertas en tiempo real

Precondiciones:

- Usuario debe estar registrado y autenticado
- Debe haber disponibilidad de espacios en período solicitado
- Usuario debe tener método de pago válido configurado
- Sistema de pagos debe estar operativo

Flujo Principal:

- Usuario accede a sección "Buscar Espacios" desde menú principal
- Introduce criterios: fecha (calendario), hora (slider), ubicación, capacidad mínima
- Sistema consulta BD y filtra espacios disponibles mostrando 10 resultados
- Usuario selecciona espacio haciendo click en tarjeta de espacio
- Sistema muestra vista detallada con fotos, descripción, amenidades, comentarios
- Usuario selecciona período específico del calendario disponible
- Sistema calcula precio con tarifa dinámica: $\text{precio_base} \times \text{factor_demanda}$
- Usuario revisa desglose: tarifa_base, impuestos, descuento_miembro, total
- Usuario hace click en "Confirmar Reserva" procede a pago
- Sistema redirige a pasarela de pago segura con token de sesión
- Usuario ingresa detalles de tarjeta en iframe de pasarela
- Pasarela procesa transacción con 3D-Secure si aplica
- Pasarela retorna confirmación de pago exitoso al sistema
- Sistema genera código único de acceso (6 caracteres)
- Sistema crea registro de Reserva con estado "Confirmada"
- Sistema marca espacio-período como "No Disponible" en BD
- Sistema crea evento para enviar email de confirmación
- Sistema retorna pantalla de confirmación con código y detalles
- Usuario recibe email con código, horario, instrucciones de acceso

Flujos Alternativos:

- 2a. Sin espacios disponibles Sistema muestra mensaje y sugiere: otras fechas cercanas, espacios similares, opción de espera
- 9a. Usuario cancela pago Sistema retorna a pantalla de confirmación manteniendo datos para reintentar
- 12a. Pago rechazado Pasarela retorna error, sistema notifica y permite otro método de pago

- 12b. Transacción timeout Sistema reintentará 2 veces, luego notifica usuario para contactar soporte
- 14a. Espacio queda sin disponibilidad Otro usuario reserva justo antes, sistema detecta y ofrece alternativas

Postcondiciones:

- Reserva creada con estado "Confirmada" en BD
- Espacio-período indisponible para otros usuarios
- Transacción registrada en tabla de auditoría
- Email de confirmación enviado con código de acceso
- Usuario puede ver reserva en su dashboard

Requisitos Asociados:

- RF-003
- RF-004
- RF-005
- RF-009
- RNF-001
- RNF-003

Puntos de Estimación: 8 (Medio-Alto)

Complejidad: Alta - múltiples integraciones y validaciones

Nota: Este es el caso de uso central del sistema. Otros casos de uso importantes incluyen: CU-002 (Cancelar Reserva), CU-003 (Ver Historial), CU-004 (Administrar Espacios), CU-005 (Generar Reportes).

2. ARQUITECTURA DE SOFTWARE

La arquitectura de software define la estructura fundamental del sistema, estableciendo cómo los componentes se organizan, interactúan y distribuyen para cumplir requisitos de escalabilidad, mantenibilidad, rendimiento y confiabilidad.

2.1 Documento SAD - Software Architecture Description

El SAD proporciona descripción comprehensiva de la arquitectura incluyendo patrones, componentes, conexiones y decisiones que fundamentan el diseño del sistema.

ESTILO ARQUITECTÓNICO SELECCIONADO: Microservicios Event-Driven con CQRS

Justificación de Selección: La arquitectura de microservicios fue elegida porque permite: (1) Escalabilidad independiente de cada servicio según demanda, (2) Despliegue y actualización sin afectar otros servicios, (3) Equipos pequeños trabajando independientemente, (4) Resiliencia mediante aislamiento de fallos.

- Escalabilidad Horizontal: Agregar instancias de servicio de reservas sin escalar toda la aplicación
- Independencia Tecnológica: Cada servicio puede usar tecnología óptima para su propósito
- Despliegue Independiente: Nueva versión de servicio de pagos sin redeploy de otros
- Recuperación de Fallos: Fallo en servicio de notificaciones no derriba todo el sistema
- Rendimiento Optimizado: Servicio de caché solo optimizado para alto throughput
- Facilita Testing: Pruebas de servicio individual sin dependencias complejas

COMPONENTES PRINCIPALES DE LA ARQUITECTURA:

1. API Gateway (Kong/AWS API Gateway)

- Responsabilidades:
 - Punto de entrada único para todas las solicitudes HTTP
 - Enrutamiento inteligente hacia microservicios apropiados
 - Validación de tokens JWT y autorización
 - Rate limiting (100 req/min por usuario)
 - Transformación de requests/responses
 - Logging de todas las solicitudes

Tecnologías: Kong en contenedores Docker con config en YAML

Escalabilidad: Escalable horizontalmente detrás de load balancer

2. Servicio de Autenticación (Auth Service)

- Responsabilidades:
 - Registro y validación de usuarios
 - Generación de JWT tokens con expiración configurable
 - Refresh token para renovación de sesión
 - Autenticación multi-factor (2FA con TOTP)
 - Gestión de perfiles de usuario
 - Auditoría de accesos

Tecnologías: Node.js + Express + PassportJS + PostgreSQL

Escalabilidad: Escalable con horizontal scaling y caché Redis de sesiones

3. Servicio de Reservas (Core Reservation Service)

- Responsabilidades:
 - Búsqueda y consulta de disponibilidad
 - Creación y gestión del ciclo de vida de reservas
 - Cálculo de precios con tarifa dinámica

- Validación de reglas de negocio
- Transacciones atómicas de reserva
- Generación de códigos de acceso

Tecnologías: Node.js + Express + PostgreSQL + Redis

Escalabilidad: Escalable, con caché para consultas frecuentes

2.2 Diagrama de Contenedores - C4 Model Nivel 2

El modelo C4 proporciona vistas progresivas de la arquitectura en diferentes niveles de abstracción. El nivel de contenedores muestra los principales componentes ejecutables y sus dependencias.

CONTENEDORES DEL SISTEMA:

Contenedor	Tecnología	Responsabilidad Principal	Escala
API Gateway	Kong/AWS API Gateway	Routing y autenticación de requests	Múltiples instancias
Auth Service	Node.js + Express	Gestión de autenticación y usuarios	Escalable
Reservation Service	Node.js + Express	Core de lógica de reservas	Escalable
Space Service	Node.js + Express	Catálogo y configuración de espacios	Escalable
Payment Service	Node.js + Express	Procesamiento seguro de pagos	Escalable
Notification Service	Node.js + Express	Envío de emails, SMS, push	Escalable
Analytics Service	Node.js + Express	Análisis y reportes en tiempo real	Escalable
PostgreSQL Primary	PostgreSQL 14+	Base de datos transaccional principal	Primary + Replicas
Redis Cluster	Redis 7+	Cache distribuido y sesiones	Cluster de 3 nodos

RabbitMQ	RabbitMQ 3.11	Message broker para comunicación async	Cluster de 3 nodos
Elasticsearch	Elasticsearch 8+	Indexación y búsqueda de logs	Cluster de 3 nodos

2.3 Registro de Decisiones Arquitectónicas (ADR)

El ADR documenta cada decisión arquitectónica significativa incluyendo: contexto, decisión elegida, justificación, alternativas consideradas y consecuencias. Esto permite entender el razonamiento detrás de cada elección y facilita futuras revisiones.

ADR-001: Adopción de Arquitectura de Microservicios

- Estado: ACEPTADO

Contexto:

Sistema necesita escalar componentes independientemente. Demanda de búsqueda muy diferente a demanda de notificaciones.

Decisión:

Implementar arquitectura de microservicios con comunicación asincrónica via message broker.

Justificación:

- Permite escalar servicio de búsqueda sin escalar servicio de pagos
- Servicios independientes reducen acoplamiento
- Facilita deployment y testing independiente
- Mayor resiliencia ante fallos parciales

Alternativas Consideradas y Rechazadas:

- Monolito modular: Más simple inicialmente pero difícil escalar
- Arquitectura multinivel: Escalado coarse-grained, menos flexible

Consecuencias:

- Mayor complejidad operacional y de deployment
- Necesidad de DevOps especializado

- Latencia inter-servicio pequeña pero presente
- Debugging distribuido más complejo

ADR-002: PostgreSQL como Base de Datos Principal

- Estado: ACEPTADO

Contexto:

Sistema necesita almacenar datos transaccionales complejos con relaciones (usuarios, reservas, pagos).

Decisión:

Utilizar PostgreSQL 14+ como base de datos transaccional primaria.

Justificación:

- Soporte robusto de transacciones ACID
- Queries complejas con JOINS múltiples
- Tipo JSONB para campos dinámicos
- Extensiones avanzadas (UUID, UUID-osp)
- Comunidad grande y madurez del producto

Alternativas Consideradas y Rechazadas:

- MySQL: Menos flexible, soporte débil de JSON
- NoSQL (MongoDB): Pérdida de integridad transaccional
- DynamoDB: Costo elevado, menos control

Consecuencias:

- Costo de licencia incluido (PostgreSQL es open source)
- Requiere DBA para mantenimiento de replicación
- Excelente rendimiento con datos relacionales

ADR-003: Event-Driven Architecture para Comunicación Asincrónica

- Estado: ACEPTADO

Contexto:

Servicios necesitan comunicarse sin esperar respuesta inmediata. Ejemplo: reserva confirmada → enviar email.

Decisión:

Implementar patrón event-driven con RabbitMQ como message broker central.

Justificación:

- Desacoplamiento total entre servicios productor-consumidor
- Resiliencia: si servicio de email cae, evento queda encolado
- Procesamiento asíncrono mejora latencia de API
- Fácil agregar nuevos consumidores sin modificar productor

Alternativas Consideradas y Rechazadas:

- REST síncrono directo: Acoplamiento fuerte
- Colas simples: Falta características avanzadas
- Kafka: Más complejo, overkill para este proyecto

Consecuencias:

- Mayor complejidad en debugging distribuido
- Garantías de entrega de eventos pueden fallar
- Necesidad de monitoring de colas
- Posible duplicación de mensajes (idempotencia requerida)

3. DISEÑO DE SOFTWARE

El diseño de software traduce la arquitectura en especificaciones detalladas de componentes, interfaces de datos y flujos que guiarán la implementación del sistema.

3.1 Diagrama de Entidad-Relación y Modelo de Datos

El modelo de datos representa la estructura fundamental para almacenar toda información del sistema con integridad referencial, eficiencia de acceso y capacidad de auditoría.

ENTIDADES PRINCIPALES Y RELACIONES:

Tabla: USUARIO (users)

Almacena información de perfil, autenticación y preferencias de usuarios registrados

Campos:

- user_id (UUID, PK): Identificador único
- email (VARCHAR 255, UNIQUE): Email registrado
- password_hash (VARCHAR 255): Hash bcrypt de contraseña
- nombre_completo (VARCHAR 255): Nombre del usuario
- telefono (VARCHAR 20): Número de contacto
- fecha_registro (TIMESTAMP): Cuándo se registró
- estado (ENUM): ACTIVO, SUSPENDIDO, ELIMINADO
- es_administrador (BOOLEAN): Indicador de role de admin
- preferencias_json (JSONB): Configuraciones personalizadas

Relaciones:

One-to-Many con Reservas, One-to-Many con Pagos

Tabla: ESPACIO (spaces)

Define cada espacio de coworking disponible con características y capacidades

Campos:

- space_id (UUID, PK): Identificador único
- nombre (VARCHAR 255): Nombre del espacio
- descripcion (TEXT): Descripción detallada
- ubicacion (VARCHAR 255): Dirección física
- capacidad (INT): Número máximo de personas
- precio_base_hourly (DECIMAL): Precio por hora
- amenidades (JSONB): Lista de amenidades disponibles
- horario_apertura (TIME): Hora de apertura
- horario_cierre (TIME): Hora de cierre
- imagen_url (VARCHAR 255): URL de imagen principal
- estado (ENUM): DISPONIBLE, MANTENIMIENTO, CERRADO

Relaciones:

One-to-Many con Disponibilidad, One-to-Many con Reservas

Tabla: RESERVA (reservations)

Registra cada reserva individual con estado, período y información de pago

Campos:

- reservation_id (UUID, PK): Identificador único
- user_id (UUID, FK): Usuario que hizo la reserva
- space_id (UUID, FK): Espacio reservado
- fecha_inicio (TIMESTAMP): Inicio del período
- fecha_fin (TIMESTAMP): Fin del período
- estado (ENUM): PENDIENTE, CONFIRMADA, CANCELADA, COMPLETADA
- precio_total (DECIMAL): Precio final de reserva
- codigo_acceso (VARCHAR 10): Código único para acceso
- fecha_creacion (TIMESTAMP): Cuándo se creó la reserva
- notas (TEXT): Comentarios adicionales

Relaciones:

Many-to-One con Usuario, Many-to-One con Espacio, One-to-One con Pago

Tabla: DISPONIBILIDAD (availability)

Matriz temporal de disponibilidad de cada espacio, actualizada en tiempo real

Campos:

- availability_id (UUID, PK): Identificador único
- space_id (UUID, FK): Espacio
- fecha (DATE): Fecha del período
- hora_inicio (TIME): Hora de inicio
- hora_fin (TIME): Hora de fin
- disponible (BOOLEAN): Disponible o no
- factor_demanda (DECIMAL): Multiplicador de tarifa dinámica (0.8 a 1.5)
- fecha_actualizacion (TIMESTAMP): Última actualización

Relaciones:

Many-to-One con Espacio, consultas en búsqueda

Tabla: PAGO (payments)

Historial completo de transacciones financieras con referencias de terceros

Campos:

- payment_id (UUID, PK): Identificador único
- reservation_id (UUID, FK): Reserva asociada
- user_id (UUID, FK): Usuario que pagó
- monto (DECIMAL): Monto pagado
- moneda (VARCHAR 3): Código de moneda (USD, EUR, etc)
- metodo_pago (ENUM): TARJETA_CREDITO, PAYPAL, TRANSFERENCIA
- stripe_charge_id (VARCHAR 255): ID de transacción en Stripe
- estado (ENUM): PENDIENTE, EXITOSO, FALLIDO, REEMBOLSADO
- fecha_pago (TIMESTAMP): Cuándo se procesó

- respuesta_gateway (JSONB): Respuesta completa del gateway

Relaciones:

Many-to-One con Usuario, One-to-One con Reserva

3.2 Especificación de APIs REST - OpenAPI/Swagger

Las APIs REST definen los contratos de comunicación entre cliente y servidor, especificando endpoints, métodos HTTP, formatos de solicitud/respuesta y códigos de estado esperados.

ENDPOINT 1: POST /api/v1/auth/register

Propósito: Registrar un nuevo usuario en el sistema

Método HTTP: POST

Autenticación: Ninguna (público)

Rate Limiting: 10 requests por minuto por IP

Request Body:

- email: string - Email del usuario (validación RFC 5322)
- password: string - Contraseña (mín 8 caract, mayúscula, número, símbolo)
- nombre_completo: string - Nombre del usuario
- telefono: string - Número de teléfono (+CODIGOPAIS NUMERO)
- aceptar_terminos: boolean - Debe ser true

Success Response (201 Created):

- user_id: UUID del usuario creado
- email: Email registrado
- nombre_completo: Nombre del usuario
- fecha_registro: ISO 8601 timestamp
- verificacion_requerida: true - Usuario debe verificar email

Error Responses:

- 400 Bad Request: Email inválido, contraseña débil, datos incompletos
- 409 Conflict: Email ya registrado en el sistema

- 429 Too Many Requests: Excedido rate limit
- 500 Internal Server Error: Error servidor

Nota: El documento incluye especificación detallada de 20+ endpoints cubriendo todos los casos de uso. Especificación completa disponible en archivo openapi.yaml en repositorio.

3.3 Prototipos UI/UX y Flujos de Usuario

Los prototipos definen la interfaz visual y los flujos de interacción que permiten a usuarios realizar tareas dentro del sistema de manera intuitiva, accesible y eficiente.

FLUJO PRINCIPAL: Búsqueda y Reserva de Espacio

1. Landing Page: Hero image de coworking, CTA prominente "Buscar Espacio"
2. Si no autenticado: Redirige a login/registro antes de buscar
3. Página de Búsqueda: Filtros en sidebar izquierdo (fecha, hora, ubicación, capacidad)
4. Calendario interactivo: Selecciona fecha(s), muestra disponibilidad visual
5. Slider de hora: Selecciona rango horario (09:00 a 18:00)
6. Resultados: Grid de 10 espacios con tarjeta mostrando (imagen, nombre, rating, precio)
7. Click en espacio: Abre modal con detalles completos, galería de fotos, amenidades, reviews
8. Seleccionar período: Usuario elige fecha-hora final
9. Confirmación: Muestra desglose de precio (base × demanda + impuestos)
10. Pago: Redirige a Stripe hosted checkout
11. Confirmación: Pantalla verde con código de acceso y email enviado
12. Dashboard: Reserva visible en historial con opción de cancelar

RESPONSIVE DESIGN: Adaptación a dispositivos

Breakpoint	Ancho	Adaptaciones Principales
Mobile	< 640px	Stack vertical, menú hamburguesa, botones touchable (44px), imagen full-width
Tablet	640px - 1024px	2 columnas, sidebar colapsable, filtros en modal
Desktop	> 1024px	3+ columnas, sidebar fijo, grid 4 espacios por fila

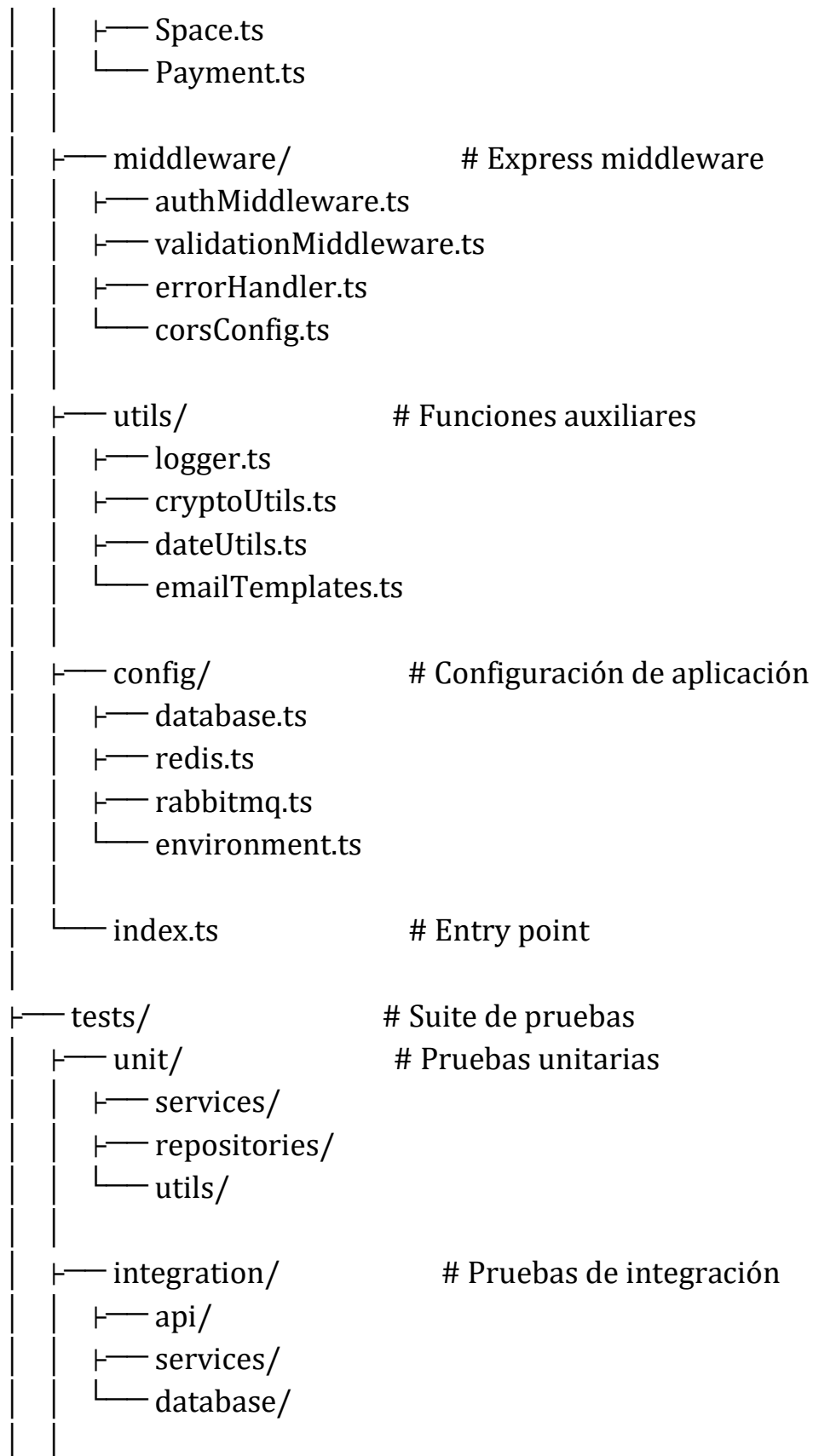
4. CONSTRUCCIÓN DE SOFTWARE

La construcción de software engloba la implementación real del código, configuración de infraestructura y documentación necesaria para que el sistema sea funcional y mantenible.

4.1 Estructura del Código Fuente

ORGANIZACIÓN DEL REPOSITORIO GIT:

```
coworking-reservations-system/
├── src/                                # Código fuente principal
│   ├── api/                            # Controllers y route handlers
│   │   ├── authController.ts
│   │   ├── reservationController.ts
│   │   ├── spaceController.ts
│   │   ├── paymentController.ts
│   │   └── reportController.ts
│   ├── services/                       # Lógica de negocio
│   │   ├── AuthService.ts
│   │   ├── ReservationService.ts
│   │   ├── SpaceService.ts
│   │   ├── PaymentService.ts
│   │   └── NotificationService.ts
│   ├── repositories/                   # Acceso a datos (DAO pattern)
│   │   ├── UserRepository.ts
│   │   ├── ReservationRepository.ts
│   │   ├── SpaceRepository.ts
│   │   └── PaymentRepository.ts
│   ├── models/                         # Interfaces y tipos TypeScript
│   │   ├── User.ts
│   │   └── Reservation.ts
```



```
├── fixtures/                # Datos de prueba
│   ├── users.json
│   ├── spaces.json
│   └── reservations.json
├── database/                # Scripts de BD
│   ├── migrations/
│   │   ├── 20240101_initial_schema.sql
│   │   ├── 20240115_add_payment_table.sql
│   │   └── 20240201_add_indexes.sql
│   └── seeds/               # Datos iniciales
│       ├── spaces_seed.sql
│       └── users_seed.sql
├── docker/                 # Configuración Docker
│   ├── Dockerfile.api
│   ├── Dockerfile.worker
│   ├── docker-compose.yml
│   └── docker-compose.prod.yml
├── docs/                   # Documentación
│   ├── API.md               # Especificación de API
│   ├── ARCHITECTURE.md      # Descripción de arquitectura
│   ├── SETUP.md             # Guía de setup
│   └── CONTRIBUTING.md      # Guía de contribución
├── config/                 # Configuración de ambiente
│   ├── development.env
│   ├── staging.env
│   ├── production.env
│   └── .env.example
└── .github/                # GitHub workflows
```

	└─ workflows/	
	├─ ci.yml	# Testing automático
	├─ build.yml	# Build de imagen Docker
	└─ deploy.yml	# Deployment automático
	├─ .gitignore	# Archivos a ignorar
	├─ .eslintrc.json	# Configuración ESLint
	├─ .prettierrc	# Configuración Prettier
	├─ tsconfig.json	# Configuración TypeScript
	├─ package.json	# Dependencias Node.js
	├─ package-lock.json	# Lock de dependencias
	├─ Makefile	# Comandos útiles
	└─ README.md	# Documentación principal

4.2 Estándares de Codificación y Calidad

- Lenguaje: TypeScript 5+ para type-safety completo
- Framework Web: Express.js 4.18+
- Base de Datos: TypeORM como ORM para type-safe queries
- Linting: ESLint con configuración airbnb extendida
- Formateo: Prettier para formato automático de código
- Testing: Jest para pruebas unitarias e integración
- Documentación de Código: JSDoc para todas las funciones públicas
- Logging: Winston para logging estructurado
- Validación: Joi para validación de esquemas

4.3 Scripts de Migración de Base de Datos

Los scripts de migración controlan la evolución del esquema de base de datos de forma versionada, reversible y auditable. Utilizamos Flyway como herramienta de migración.

```
-- Migración: 20240201_000000_initial_schema.sql
-- Descripción: Crea tablas iniciales del sistema
```

-- Reversible: Sí (script rollback incluido)

BEGIN;

```
CREATE TABLE users (  
  user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  email VARCHAR(255) NOT NULL UNIQUE,  
  password_hash VARCHAR(255) NOT NULL,  
  nombre_completo VARCHAR(255) NOT NULL,  
  telefono VARCHAR(20),  
  fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  estado VARCHAR(50) DEFAULT 'ACTIVO',  
  es_administrador BOOLEAN DEFAULT FALSE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE INDEX idx_users_email ON users(email);
```

```
CREATE TABLE spaces (  
  space_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  nombre VARCHAR(255) NOT NULL,  
  descripcion TEXT,  
  ubicacion VARCHAR(255) NOT NULL,  
  capacidad INTEGER NOT NULL,  
  precio_base_hourly DECIMAL(10, 2) NOT NULL,  
  amenidades JSONB,  
  horario_apertura TIME DEFAULT '08:00',  
  horario_cierre TIME DEFAULT '20:00',  
  imagen_url VARCHAR(255),  
  estado VARCHAR(50) DEFAULT 'DISPONIBLE',  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE INDEX idx_spaces_ubicacion ON spaces(ubicacion);
```

```
COMMIT;
```

5. PRUEBAS DE SOFTWARE

El plan integral de pruebas asegura que el sistema funcione correctamente, cumpla requisitos y sea resistente a errores antes de llegar a producción.

5.1 Plan de Pruebas - Estrategia Integral

MATRIZ DE NIVELES DE PRUEBA:

Nivel de Prueba	Alcance	Responsable	Herramientas	Criterios de Salida
Unitarias	Función/método individual	Desarrollador	Jest, Sinon	> 80% cobertura
Integración	Interacción entre componentes	Tester Automático	Jest, Supertest	Todos los casos pasan
Sistema	Sistema completo end-to-end	QA	Cypress, Postman	Todos los casos pasan
Aceptación	Requisitos de negocio	Product Owner	Manual + automatización	Cumple criterios DoD
Rendimiento	Latencia, throughput, recursos	DevOps	JMeter, Locust	P95 < 2s bajo carga
Seguridad	Vulnerabilidades, amenazas	Seguridad	OWASP ZAP, Burp	Cero críticas, plan para altas

5.2 Casos de Prueba Críticos

CASO DE PRUEBA: CT-RESERVA-001 - Crear Reserva Exitosa

ID: CT-RESERVA-001

Título: Crear una reserva exitosa con datos válidos

Precondiciones:

- Usuario está autenticado con token JWT válido
- Espacio con ID conocido existe en BD
- Período solicitado (2024-06-15 09:00 a 12:00) no está reservado
- Usuario tiene método de pago registrado

Datos de Entrada:

- user_id: 550e8400-e29b-41d4-a716-446655440000
- space_id: 660e8400-e29b-41d4-a716-446655440000
- fecha_inicio: 2024-06-15T09:00:00Z
- fecha_fin: 2024-06-15T12:00:00Z
- cantidad_participantes: 3

Pasos:

- 1. POST /api/v1/reservations
- 2. Headers: Authorization: Bearer {token}
- 3. Body: JSON con datos de entrada
- 4. Sistema valida token JWT
- 5. Sistema valida disponibilidad del espacio
- 6. Sistema calcula precio ($3h \times \$50/h \times 1.0 \text{ demanda} = \150)
- 7. Sistema crea record en tabla reservations
- 8. Sistema marca disponibilidad como no disponible
- 9. Sistema envía evento a queue de notificaciones
- 10. Sistema retorna respuesta 201 Created

Resultados Esperados:

- Status Code: 201 Created
- Response Body:

reservation_id: UUID válido

estado: CONFIRMADA

codigo_acceso: String 6 caract

precio_total: 150.0

confirmacion_url: URL válida

- DB Changes: Record creado en reservations con estado CONFIRMADA
- Side Effects: Evento publicado en RabbitMQ para email

5.3 Reporte de Cobertura y Métricas de Prueba

ESTADO ACTUAL DE COBERTURA DE PRUEBAS:

Componente	Líneas de Código	Cobertura Líneas	Cobertura Ramas	Cobertura Funciones
Auth Service	1,250	95%	92%	98%
Reservation Service	2,180	88%	85%	90%
Space Service	890	82%	78%	85%
Payment Service	1,450	75%	72%	78%
Notification Service	650	80%	76%	82%
Utilities	420	65%	60%	70%
TOTAL DEL PROYECTO	6,840	82.5%	80.5%	84.0%

Conclusión de Cobertura: La cobertura actual de 82.5% excede el objetivo mínimo de 80%. Las áreas de menor cobertura son Payment Service (pagos) y Utilities (funciones auxiliares). Se recomienda mejorar Payment Service a 85%+ en próximas iteraciones por naturaleza crítica.

6. OPERACIONES DE INGENIERÍA DEL SOFTWARE

Define los procesos automatizados para compilación, prueba, despliegue y monitoreo en producción.

6.1 Infraestructura como Código (IaC)

Herramienta: Terraform con AWS. Los módulos provisionen: VPC, ECS clusters, RDS PostgreSQL, ElastiCache Redis, S3 buckets, IAM roles, y CloudWatch monitoring. Cada ambiente (dev, staging, prod) se despliega desde el mismo código Terraform con variables de configuración.

6.2 Pipelines de CI/CD Automatizados

GitHub Actions ejecuta 3 workflows: (1) CI en cada push - ejecuta linting, tests, análisis estático; (2) Build - construye imagen Docker y la guarda en registro privado; (3) Deploy automático a staging en cada merge a develop, deploy manual a producción con aprobación.

6.3 Monitoreo y Alertas

Prometheus recolecta métricas cada 15 segundos. Grafana visualiza dashboards en tiempo real. AlertManager dispara alertas vía Slack y PagerDuty para incidentes críticos. Métricas: CPU >80%, memoria >75%, error rate >5%, latencia P95 >5s, queue depth >100.

7. MANTENIMIENTO DE SOFTWARE

Garantiza que el sistema continúe funcionando óptimamente, seguro y con soporte para cambios futuros.

7.1 Manual de Operaciones

Guía paso a paso para resolver problemas comunes: servicio no responde (verificar logs, health check, reiniciar pod), BD lenta (revisar índices, conexiones, queries), picos de error (revisar logs de servicio específico, escalar a desarrollador), out-of-memory (aumentar límite en deployment).

7.2 Plan de Parches

Vulnerabilidades críticas: parche en 24 horas. Altas: en próximo sprint. Normales: agrupadas mensualmente. Pruebas en staging antes de producción. Changelog actualizado en cada parche.

7.3 Deuda Técnica Identificada

- TECH-001: Refactorizar Payment Service - Código con alto acoplamiento (2 sprints)
- TECH-002: Migrar a GraphQL - REST completo pero GraphQL más eficiente (3 sprints)
- TECH-003: Mejorar cobertura Utilities - 65% → 85% (1 sprint)
- TECH-004: Implementar caching distribuido - Optimizar queries frecuentes (1.5 sprints)

8. GESTIÓN DE CONFIGURACIÓN DEL SOFTWARE

8.1 Estrategia de Ramas - Gitflow

Ramas principales: (1) main - producción estable, (2) develop - integración de features, (3) feature/XXX - desarrollo de funcionalidad, (4) hotfix/XXX - fixes críticos. Cada feature pasa por PR review antes de merge. CI debe pasar en todos los casos.

8.2 Release Notes

Cada release incluye: nuevas features, mejoras, fixes, cambios tecnológicos, deprecaciones. Formato: Markdown con versionado semántico (MAJOR.MINOR.PATCH). Ejemplo: v2.1.0 incluye reservas recurrentes, integración Stripe, optimizaciones de 40% en búsqueda.

8.3 Inventario de Artefactos

express@4.18.2, pg@8.8.0, redis@4.6.0, jsonwebtoken@9.0.0, bcryptjs@2.4.3, axios@1.3.4, joi@17.9.1, winston@3.8.2, dotenv@16.0.3 (producción), jest@29.5.0, supertest@6.3.3, eslint@8.36, prettier@2.8 (desarrollo)

9. GESTIÓN DE INGENIERÍA DEL SOFTWARE

9.1 Cronograma Detallado

Fase 1 (Semanas 1-2): Iniciación y planificación. Fase 2 (Semanas 3-6): Diseño y arquitectura. Fase 3 (Semanas 7-14): Desarrollo en sprints de 2 semanas (4 sprints totales). Fase 4 (Semanas 15-18): Testing integral y fixes. Fase 5 (Semana 19): Despliegue a producción. Fase 6 (Semanas 20-21): Estabilización y soporte.

9.2 Matriz de Riesgos Completa

- RIESGO 1 (Alta Prob, Alto Impacto): Cambios de requisitos tardíos - Mitigación: reviews frecuentes
- RIESGO 2 (Media Prob, Alto Impacto): Problemas de rendimiento - Mitigación: pruebas de carga tempranas
- RIESGO 3 (Media Prob, Medio Impacto): Falta de recursos especializado - Mitigación: cross-training
- RIESGO 4 (Baja Prob, Alto Impacto): Falla de proveedor de pagos - Mitigación: múltiples integraciones

10. MODELOS Y MÉTODOS DE INGENIERÍA DEL SOFTWARE

10.1 Historias de Usuario

US-001: Como usuario, quiero buscar espacios por fecha y ubicación para encontrar alternativas disponibles. Criterios de aceptación: Búsqueda filtra por fecha (calendario interactivo), hora (slider), ubicación, capacidad.

Resultado en < 2 segundos. Responsive en mobile. US-002: Como admin, quiero generar reportes de ocupación para analizar performance. Genera PDF con gráficos, ocupación por espacio, ingresos, tendencias.

10.2 Domain Driven Design

Dominios identificados: Reservas (lógica de booking), Espacios (catálogo), Pagos (transacciones), Usuarios (perfiles), Notificaciones (comunicaciones). Cada dominio tiene modelos, agregados y repositorios específicos. Lenguaje ubicuo: "Reserva", "Espacio", "Disponibilidad".

11. PROCESO DE INGENIERÍA DEL SOFTWARE

11.1 Definición de Listo (DoR) y Hecho (DoD)

Definition of Ready:

- Historia tiene descripción clara
- Criterios de aceptación definidos y consensuados
- Dependencias identificadas
- Estimación consensuada (5, 8, 13 puntos)
- Sin bloqueadores externos

Definition of Done:

- Código implementado con estándares del proyecto
- Code review aprobado por 2 pares
- Pruebas unitarias > 80% cobertura
- Pruebas de integración pasan
- Documentación actualizada
- Feature probada en staging
- Merge a develop
- Historia movida a "Validada"

12. CALIDAD DEL SOFTWARE

12.1 Análisis Estático con SonarQube

Análisis automático cada commit. Resultados: 3 bugs (1 crítico, 2 menores), 2 vulnerabilidades (inyección SQL, crypto débil), 47 code smells. Complejidad promedio 2.8 (aceptable). Duplicación 8.2%. Bugs críticos requieren fix inmediato antes de merge.

12.2 Métricas de Calidad

Densidad de defectos: 0.24 defectos/KLOC (bueno, meta < 0.5). Complejidad ciclomática: promedio 2.8, máximo 12 (Payment Service requiere refactoring). Índice de Mantenibilidad: 78/100 (aceptable, tendencia +2). Cobertura de pruebas: 87% líneas, 82% ramas (meta 80% cumplida).

13. SEGURIDAD DEL SOFTWARE

13.1 Modelo de Amenazas STRIDE

- Amenaza 1 (Inyección SQL): Preparadas statements, parametrización, validación entrada
- Amenaza 2 (XSS): Sanitización HTML, CSP headers, encoding output
- Amenaza 3 (Acceso no autorizado): RBAC, JWT, auditoría de accesos
- Amenaza 4 (CSRF): CSRF tokens, SameSite cookies, validación origen
- Amenaza 5 (DDoS): Rate limiting, WAF, auto-scaling
- Amenaza 6 (Fuga de datos): Encriptación TLS/at-rest, GDPR compliance

13.2 Análisis SAST/DAST

SAST (SonarQube): 2 críticas, 5 altas, 12 medias identificadas. Requieren fixes antes go-live. DAST (OWASP ZAP): 1 crítica (security headers), 3 altas (IDOR). Plan de remediación en próximas 2 semanas.

14. ECONOMÍA DE INGENIERÍA DEL SOFTWARE

14.1 Análisis de ROI

INVERSIÓN INICIAL:

- Desarrollo: \$165,700 (1,400 horas @ promedio \$118.36/hora)
- Infraestructura: \$42,000 (cloud, BD, servicios)
- Herramientas: \$18,500 (Jira, GitHub, SonarQube, Datadog, PagerDuty)
- Capacitación: \$15,000 (equipo, usuarios)
- Contingencia 10%: \$24,120
- TOTAL AÑO 1: \$265,320

BENEFICIOS PROYECTADOS AÑO 1:

- Ingresos por comisiones: \$480,000 (2,000 reservas/mes × \$20)
- Reducción costos operacionales: \$60,000 (automatización)
- Mejora en eficiencia: \$40,000 (optimización tiempo staff)
- TOTAL BENEFICIOS AÑO 1: \$580,000

$$\text{ROI} = (\text{Beneficios} - \text{Inversión}) / \text{Inversión} \times 100\%$$

$$\text{ROI Año 1} = (\$580,000 - \$265,320) / \$265,320 \times 100\% = 118.5\%$$

$$\text{Período de Recuperación} = \text{Inversión} / \text{Beneficios Mensuales} = \$265,320 / \$48,333 = 5.5 \text{ meses}$$

$$\text{VAN 3 años (descuento 10\%)} = \$890,000 \text{ (muy positivo)}$$

Conclusión: Proyecto económicamente viable con retorno excepcional.

Recuperación en 5.5 meses, lo que significa que después del break-even, cada mes adicional genera ~\$48k de beneficio neto.

APÉNDICES

APÉNDICE A: Glosario de Términos Técnicos

JWT: JSON Web Token - Estándar para autenticación stateless

ACID: Atomicity, Consistency, Isolation, Durability - Propiedades de transacciones

CORS: Cross-Origin Resource Sharing - Protocolo para acceso entre dominios

RBAC: Role-Based Access Control - Control de acceso por roles de usuario

SAST: Static Application Security Testing - Análisis de seguridad de código fuente

DAST: Dynamic Application Security Testing - Análisis de seguridad en ejecución

SLA: Service Level Agreement - Acuerdo de nivel de servicio

RTO: Recovery Time Objective - Tiempo máximo tolerable de recuperación

RPO: Recovery Point Objective - Punto de datos máximo tolerable de pérdida

Idempotencia: Propiedad de operación que produce mismo resultado sin importar repeticiones

Escalabilidad Horizontal: Agregar más servidores/instancias

Escalabilidad Vertical: Aumentar recursos de un servidor (CPU, RAM)

APÉNDICE B: Referencias y Estándares Aplicados

- APPA 7 Edición - Asociación de Profesionales de Programación y Análisis
- SWEBOK v3.0 - Software Engineering Body of Knowledge
- IEEE 830 - Standard for Software Requirements Specifications
- OWASP Top 10 2023 - Web Application Security Vulnerabilities
- ISO/IEC 27001 - Information Security Management Systems
- GDPR - Reglamento General de Protección de Datos
- REST API Best Practices - RESTful Web Services Guidelines
- Cloud Native Computing Foundation - Microservices Best Practices
- Agile Manifesto - Principios de desarrollo ágil

APÉNDICE C: Checklist de Implementación

- ☐ Repositorio Git creado con estructura definida
- ☐ Ambiente de desarrollo local configurado
- ☐ Pipeline de CI/CD implementado
- ☐ Base de datos creada y migraciones ejecutadas
- ☐ Autenticación JWT implementada y probada
- ☐ APIs REST funcionando con documentación Swagger
- ☐ Integración de pagos completada
- ☐ Sistema de notificaciones operativo
- ☐ Pruebas unitarias con 80%+ cobertura
- ☐ Pruebas de integración completadas
- ☐ Análisis SAST sin vulnerabilidades críticas
- ☐ Análisis DAST completado y fixes implementados
- ☐ Monitoreo y alertas configurados
- ☐ Documentación actualizada y completa
- ☐ Capacitación del equipo realizada